Desenvolvimento Web no lado servidor – Introdução ao PHP 2025/1

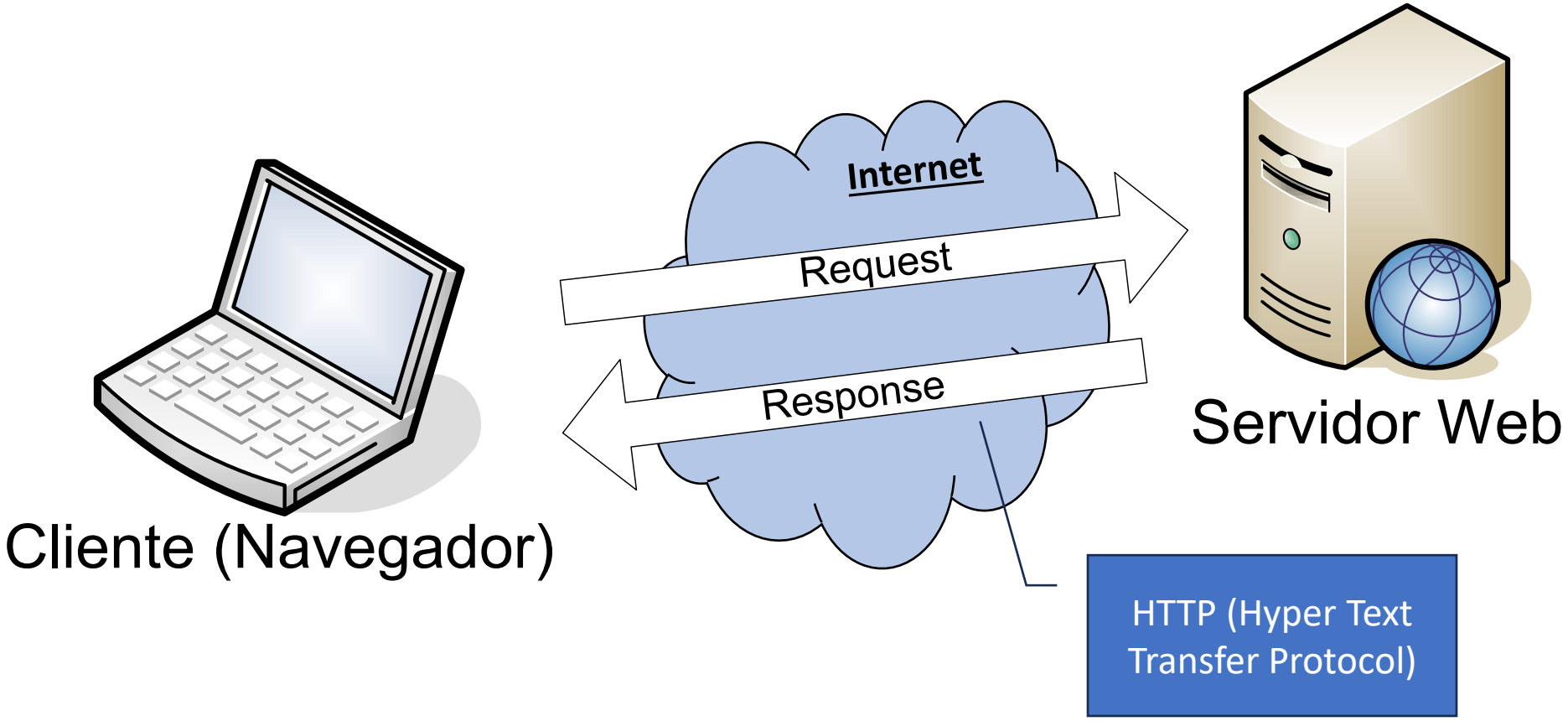
FACOM32603 - Desenvolvimento Web I

Prof. Ian R. Cunha

ianresende@ufu.br

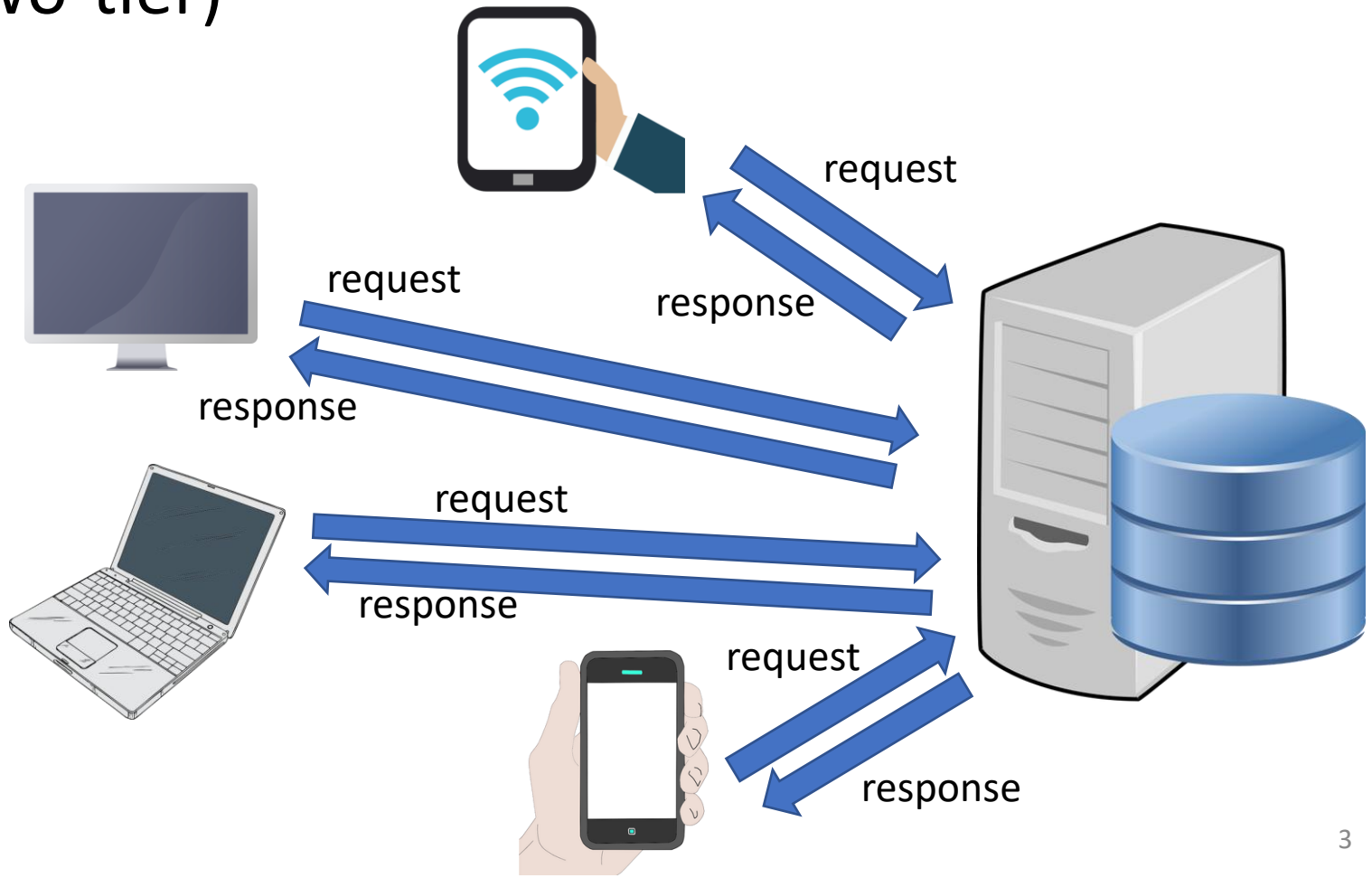
Universidade Federal de Uberlândia

Modelo de arquitetura Cliente/Servidor



Arquitetura em camadas

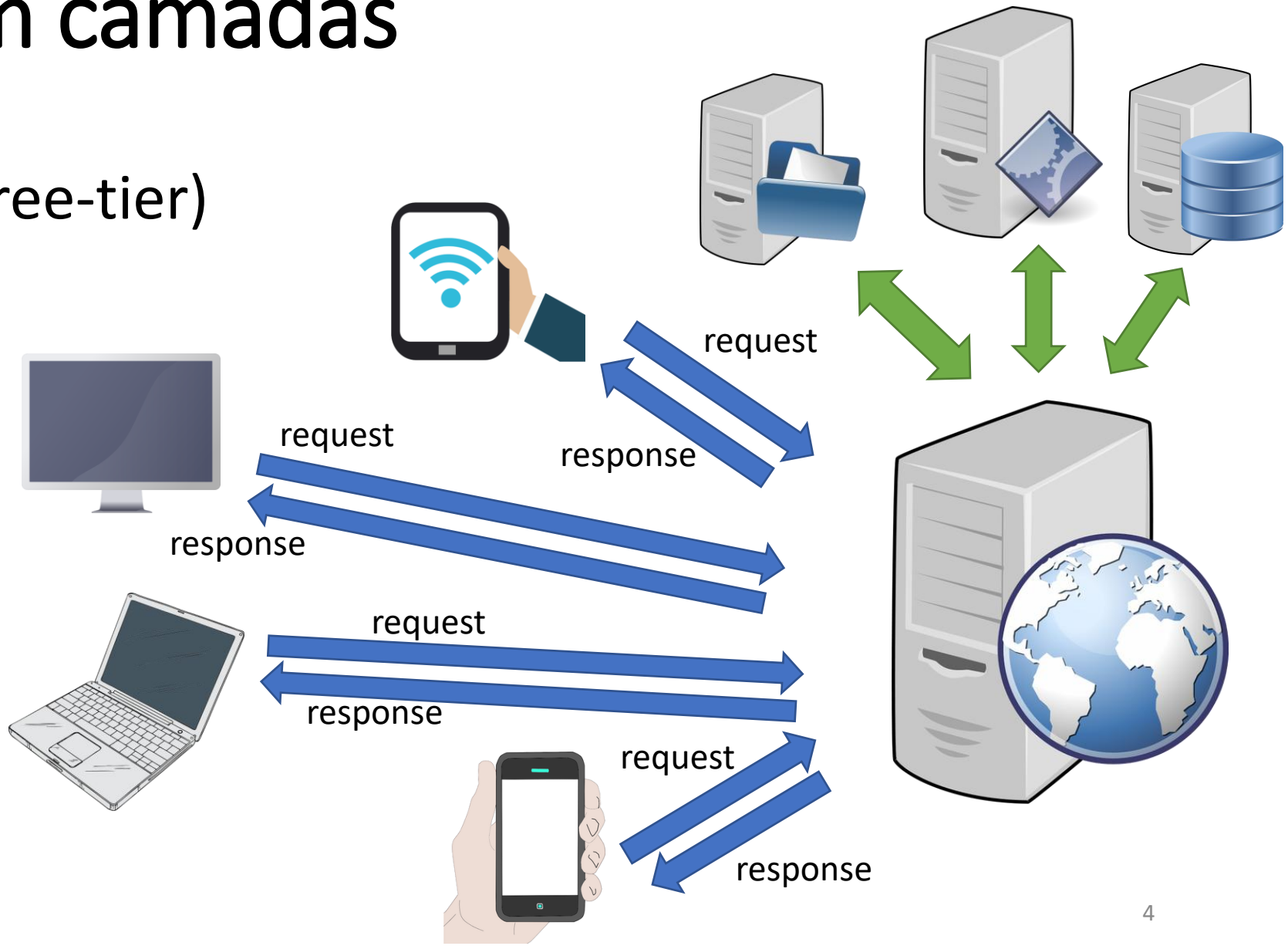
- Duas camadas (two-tier)



Imagens: https://publicdomainvectors.org

Arquitetura em camadas

- Três camadas (three-tier)



Imagens: https://publicdomainvectors.org

Protocolo HTTP

HTTPS: tráfego seguro de dados. Utiliza o protocolo SSL para criptografar os dados.

- É um protocolo que os clientes e os servidores usam para se comunicar.
  - *Hypertext Transfer Protocol* (Protocolo de Transferência de Hipertexto)
- Essa comunicação é baseada em requisições (*request*) e respostas (*responses*).
- Usado na WWW desde 1990.
  - Versão atual: HTTP/1.1

Principais métodos HTTP

- GET - Solicita ao servidor um recurso chamado de solicitação URL. Este é o método padrão, pois é a forma como o browser chama o servidor quando digita-se uma URL para que ele a recupere.
- POST - Contém um corpo nos quais seus parâmetros de solicitação já são codificados. O mais frequente uso desse método é na submissão de formulários.
- PUT - Esse método permite atualização de dados no servidor Web.
- DELETE - Permite a exclusão dentro do servidor Web.
- Além desses: HEAD, TRACE, OPTIONS, PATCH e CONNECT

HTTP requests

Linha de requisição
Cabeçalho (zero ou mais campos)

Corpo da mensagem (opcional)

Linha em branco indicando o fim do cabeçalho

```
GET /hello.html HTTP/1.1
User-Agent: Mozilla/4.0 (compatible;
MSIE5.01; Windows NT)
Host: www.exemplo.com
```

```
POST /login.html HTTP/1.1
Content-Type: application/j
Content-Length: 41
User-Agent: Mozilla/4.0 (compatible;
MSIE5.01; Windows NT)
Host: www.exemplo.com

{
        "usuario": "valor1",
         "senha": "valor2"
}
```

MIME type

HTTP responses

Linha de status
Cabeçalho (zero ou mais campos)

Corpo da mensagem (opcional)

Linha em branco indicando o fim do cabeçalho

HTTP/1.1 200 OK
Date: Mon, 11 Mar 2019 15:00:00 GMT
Content-Length: 88
Content-Type: text/html

```
<html>
<body>
<h1>Hello, World!</h1>
</body>
</html>
```

HTTP Status Codes

- **1xx: Informativa**
  - Solicitação recebida, dando continuidade ao processo. Indica uma resposta provisória
  - Exemplo: 100 (Continuar): indica que o servidor recebeu os cabeçalhos da solicitação e que o cliente deve proceder com o envio do corpo do pedido

HTTP Status Codes

- **2xx: Sucesso**
  - Solicitação recebida, compreendida, aceita e processada com êxito
  - Exemplo: 200 (OK): requisição correta

HTTP Status Codes

- **3xx: Redirecionamento**
  - O cliente deve tomar medidas adicionais para completar o pedido.
  - Exemplo: 301 (Movido): esta e todas as solicitações futuras devem ser direcionadas para o URI

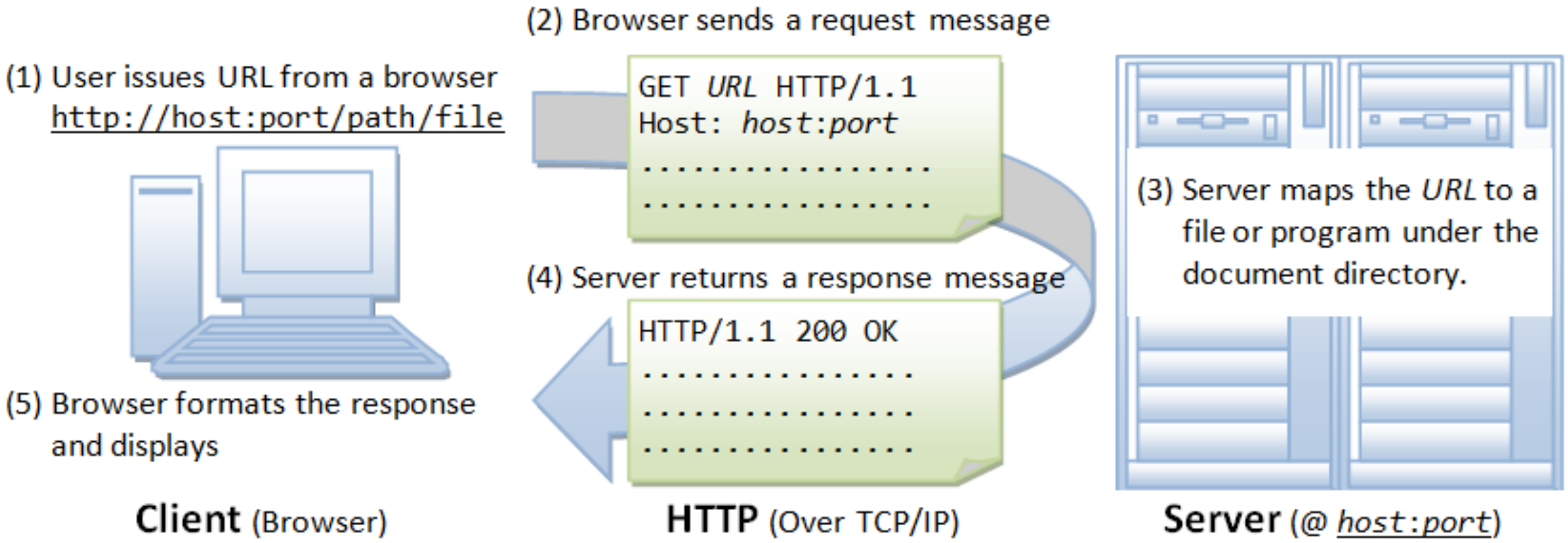
HTTP Status Codes

- **4xx: Erro de cliente**
  - O cliente fez uma requisição errada.
  - Exemplos:
    - 400 (Requisição inválida - Bad request): o servidor não entendeu a requisição
    - 401 (Não autorizado): o recurso solicitado exige usuário e senha para acesso
    - 404 (Não encontrado): O recurso solicitado não foi encontrado neste momento

HTTP Status Codes

- **5xx: Erro de servidor**
  - A requisição causou um erro no servidor.
  - Exemplos:
    - 500 (Erro interno do servidor - Internal Server Error): a requisição não foi completada porque o servidor encontrou uma situação inesperada
    - 503 (Serviço indisponível - Service Unavailable): o servidor está em manutenção ou não consegue dar conta dos processamentos de recursos devido à sobrecarga do sistema

Fluxo HTTP



https://www.ntu.edu.sg/home/ehchua/programming/webprogramming/HTTP_Basics.html

Servidores Web

- É um software responsável por aceitar solicitações e respondê-las corretamente
- O software é executado e fica disponível a espera de conexões dos clientes
- Os sistemas Web implementados são hospedados por este servidor (*host*) para que ele consiga tratar as solicitações
  - Um servidor pode hospedar mais de um sistema

Tipos de servidores

- Servidor HTTP (conhecido como servidor Web): envia os arquivos que compõem um site.
- Servidor FTP: realiza upload e download de arquivos entre computadores e servidores.
- Servidor de e-mail: envia, recebe e armazena e-mails.
- Servidor de banco de dados: armazena dados em uma estrutura específica.

Servidores Web HTTP

- Páginas estáticas: o servidor já possui o documento criado e simplesmente retorna para o cliente
  - Website que vínhamos fazendo na disciplina
- Páginas dinâmicas: o servidor realiza um processamento baseado na requisição para criar o documento a ser retornado para o cliente
  - Precisa de uma linguagem de programação

Servidores Web mais populares[1]

- Apache
  - open source e de utilização gratuita
  - desenvolvido pela Apache Software Foundation desde 1995
  - Um dos mais utilizados no mundo
  - escrito em C
  - suporte CGI, FastCGI, SCGI, uWSGI
  - algumas linguagens suportadas: PHP, Perl, Python

CGI (*Common Gateway Interface):* tecnologia que permite gerar páginas dinâmicas

Servidores Web mais populares

- IIS (*Internet Information Services*)
  - software proprietário
  - desenvolvido pela Microsoft
  - distribuído juntamente com o Windows Server
  - linguagens suportadas: ASP, ASP.NET, PHP
  - escrito em C++

Servidores Web mais populares

- Nginx (lê-se “engine x”)
  - open source e de utilização gratuita
  - alta performance
  - desenvolvido por Igor Sysoev (engenheiro de software russo)
  - escrito em C
  - suporte a FastCGI, uWSGI, SCGI

Servidores Web mais populares

- Apache Tomcat
  - servidor web Java (Java Servlet Container)
  - open source
  - desenvolvido pela Apache Software Foundation
  - escrito em Java
  - suporte ao Java Enterprise Edition (J2EE)

Servidores Web mais populares

- node.js
  - ambiente de execução Javascript server-side (ou seja, fora de um browser)
  - open source
  - escrito em C, C++ e JavaScript
  - altamente escalável

Introdução ao PHP

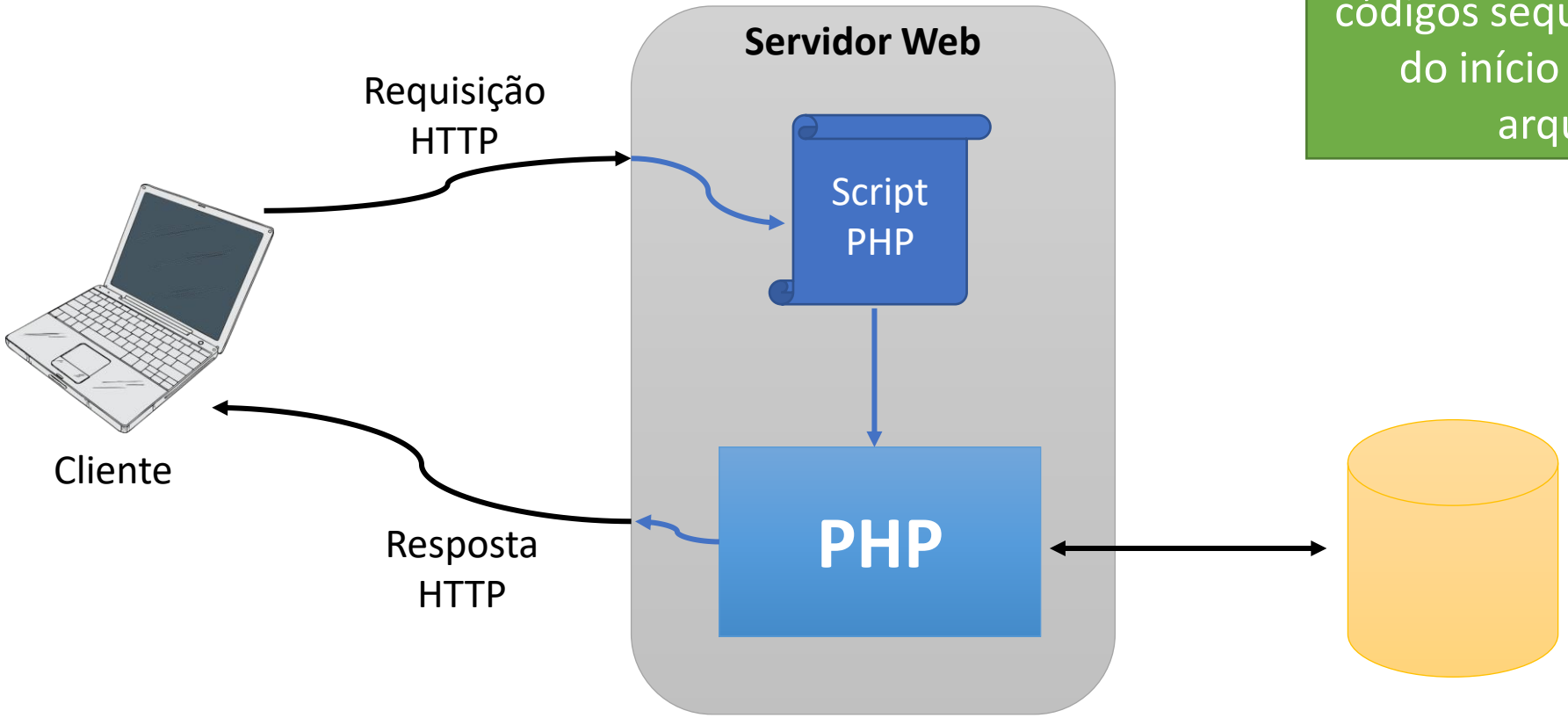
- PHP (*Hypertext Preprocessor*, originalmente *Personal Home Page Tools*) é uma linguagem de programação interpretada utilizada para desenvolver aplicações Web dinâmicas que são executadas no lado servidor

A aplicação resultante atua no tratamento das requisições HTTP feitas pelo navegador.

Introdução ao PHP

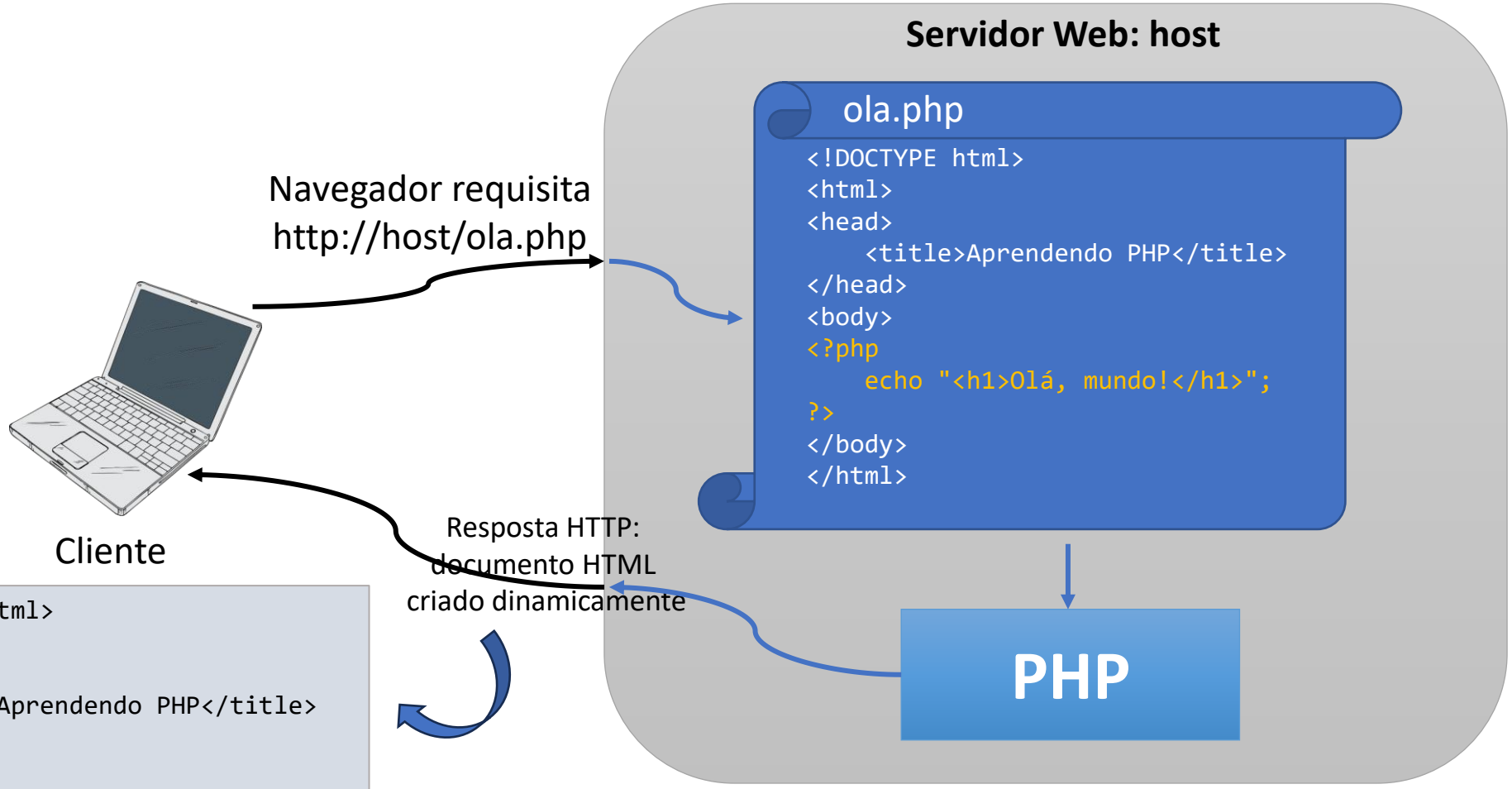
- É possível interagir com **bancos de dados** e **aplicações** existentes no servidor, com a vantagem de não expor o código fonte para o cliente
- É multiplataforma, tendo suporte aos sistemas operacionais mais utilizados no mercado
- Software gratuito e de código aberto
- Já está na versão 8.5
- https://www.php.net/

Processamento de requisições HTTP



Script PHP executa seus códigos sequencialmente do início ao fim do arquivo.

Processamento de requisições HTTP



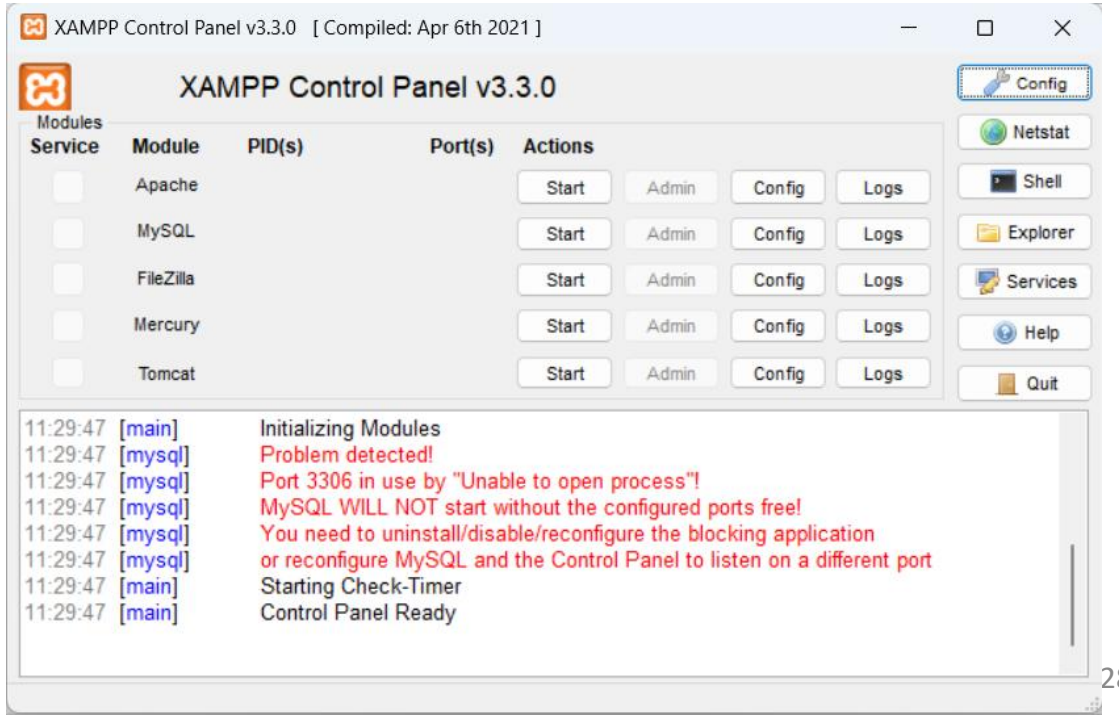

```
<!DOCTYPE html>
<html>
<head>
    <title>Aprendendo PHP</title>
</head>
<body>
<h1>Olá, mundo!</h1>
</body>
</html>
```

Preparando o ambiente

- Antes de criar uma aplicação PHP, é necessário ter um servidor Web HTTP com suporte para a linguagem PHP para hospedar sua aplicação
  - Por exemplo, Apache
- Um editor de texto (ou IDE, ambiente integrado de desenvolvimento) para criação dos scripts
  - Visual Studio Code, PHPStorm, Atom, Nuclide, Netbeans, Eclipse, Sublime, Notepad++, ...

Preparando o ambiente

- XAMPP é um ambiente de desenvolvimento que facilita a instalação de alguns servidores de código aberto
  - como FTP (transferência de arquivos)
  - banco de dados MySQL
  - Apache com suporte às linguagens PHP e Perl
- Possui um painel de controle



https://www.apachefriends.org/pt_br/index.html

Primeiros passos

- Os documentos HTML serão colocados dentro de arquivos .php pra que possam ser interpretados pelo servidor
  - Ou seja, o código PHP fica embutido no HTML dentro da tag:

```
<?php
    //Código PHP aqui!
?>
```

Delimitador

Hello, World!

Atenção! Para executar o script, basta colocá-lo na pasta do servidor e abrir no navegador: http://localhost/ola.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Aprendendo PHP</title>
</head>
<body>
<?php
    echo "<h1>Olá, mundo!</h1>";
?>
</body>
</html>
```

Script PHP

“echo” : Imprime strings e não possui retorno

Exercício

- Instalar o XAMPP em seu notebook (ou computador)
- Ativar o serviço Apache
- Criar um arquivo “hello.php” na pasta htdocs
- No arquivo criar uma estrutura html básica e inserir uma frase html por meio da tag php: <?php ?>
- Executar o arquivo: http://localhost/hello.php

Declaração de variáveis

- Não é necessário declarar o tipo da variável
  - Tipagem dinâmica (assume um tipo de acordo com o valor que lhe é atribuído)
- Sempre começam com $ e então o nome da variável
  - Exemplos: $v1, $total
- Diferencia minúsculo de maiúsculo (*case-sensitive*)

Declaração de variáveis

- string
  - `$nome = "Fulano";`
  - Pode-se usar aspas duplas ou simples
- int
  - `$idade = 20;`
- float / double
  - `$salario = 1000.00;`
- bool
  - `$ativo = true;`

Variáveis - exemplo

```php
<?php
    $nome1 = "João";
    $nome2 = "Maria";
    echo "$nome1, $nome2"; //Exibe: "João, Maria"
?>
```

O conteúdo da variável é colocado no lugar.

Comentários

```
// Comentário de linha

# Comentário de linha

/*Comentário de mais
de uma linha
*/
```

Constantes

- São dados que possuem valores que não mudam
- Exemplo:
  - `define('MAX', 10); //define uma constante chamada de MAX com o valor 10`
- Para ler o valor de uma constante, basta utilizar o seu nome como uma variável (sem o $)

Operadores aritméticos

Nome	Exemplo
Adição	$a + $b
Subtração	$a - $b
Multiplicação	$a * $b
Divisão	$a / $b
Resto	$a % $b

Concatenação de strings

- Para concatenar duas ou mais strings, basta utilizar o sinal de ponto (.)
- Exemplo:

```
$a = "Hello ";
$b = $a . "World!";
```

O valor da variável $b será “Hello World!”

Operadores lógicos

Nome	Exemplo
E	$a && $b
OU	$a || $b
NOT	! $a

Operadores de comparação

Nome	Operador
Igual a	==
Igual a (inclusive tipo)	===
Diferente	!= <>
Maior que	>
Maior ou igual	>=
Menor	<
Menor ou igual	<=

Controle de fluxo

- Da mesma forma que em outras linguagens de programação, o PHP oferece a possibilidade de efetuar
  - Controle condicional com if..else e switch..case
  - Controle de repetição com for, while, do..while, foreach

Controle de fluxo

Condicional

```
if($nome == "Teste123"){
    echo "Válido";
} else {
    echo "Inválido";
}
```

```
switch($valor){
    case "1":
        echo "bom dia";
        break;
    case "2":
        echo "boa tarde";
        break;
    default:
        echo "boa noite";
}
```

Repetição

```
$count = 1;
while($count <= 10){
    echo $count;
    $count++;
}
```

```
for($i = 0; $i < 10; $i++){
    echo "Número" . $i;
}
```

```
$i = 1;
do{
    echo "Número " . $i;
    $i++;
}while($i <= 10);
```

Arrays

- Na verdade é um mapa ordenado, ou seja, um conjunto que relaciona chaves e valores.
  - A **chave** pode ser um inteiro ou uma string.
- Pode ser criado com o construtor de linguagem array()
- Exemplo:

```
$meuArray = array(
    “chave1” => “valor1”,
    “chave2” => “12345”
);
```

```
$meuArray = [
    ”foo” => ”valor1”,
    ”bar” => ”12345”
];
```

Arrays

- Não é necessário ter chave. Nesse caso, o acesso é dado pelo índice.
- Exemplo:

```
$meuArray = array( “valor1”, “valor2” );
```

```
$meuArray = [ “valor1”, “valor2” ];
```

Arrays

- O acesso aos valores de um array pode ser pelo índice ou pela chave.
- Exemplos:
  - `$meuArray[0]`: somente para os casos que não possuem chaves
  - `$meuArray["chave1"]`

Algumas funções prontas para Arrays

Contar e Verificar

- **count($array):** Retorna o número de elementos no array.
- **in_array($valor, $array):** Verifica se um valor existe no array. (true ou false)
- **array_key_exists($chave, $array):** Verifica se uma chave existe no array.

Adicionar e Remover Elementos

- **array_push($array, $valor):** Adiciona um ou mais elementos no final do array. (É mais comum e um pouco mais rápido usar $array[] = $valor;).
- **array_pop($array):** Remove e retorna o último elemento do array.
- **array_unshift($array, $valor):** Adiciona um ou mais elementos no início do array.
- **array_shift($array):** Remove e retorna o primeiro elemento do array.
- **unset($array[chave]):** Remove um elemento específico pela sua chave.

Algumas funções prontas para Arrays

Ordenar Arrays

- **sort()** Ordena pelos valores (crescente). (Não mantém chaves)
- **rsort()** Ordena pelos valores (decrescente). (Não mantém chaves)
- **asort()** Ordena pelos valores, mantendo a associação chave-valor.
- **arsort()** Ordena pelos valores (decrescente), mantendo as chaves.
- **ksort()** Ordena pelas chaves, mantendo a associação chave-valor.
- **krsort()** Ordena pelas chaves (decrescente), mantendo as chaves.

Foreach

```php
$nomes = array("Ana", "Caio");
foreach($nomes as $valor){
    echo "Nome atual: " . $valor . "<br/>";
}
```

```php
for($i = 0; $i < sizeof($nomes); $i++){
    echo "Nome atual: " . $nomes[$i] . "<br/>";
}
```

Funções

Nome único

Qualquer quantidade de parâmetros. Pode não ter parâmetros.

```
<?php
function foo ($arg_1, $arg_2, /* ..., */ $arg_n)
{
    echo "Exemplo de função.\n";
    return $valor_retornado;
}
?>
```

Algumas funções prontas para Strings

- strtoupper: converte a string para letras maiúsculas
- strtolower: converte a string para letras minúsculas
- str_replace: substitui todos os caracteres de um tipo de por outro em uma string
- strlen: conta quantos caracteres
- str_word_count: conta quantas palavras
- strip_tags: retira as tags HTML do texto. Opcionalmente, pode-se permitir que algumas delas continuem no texto.

Algumas funções prontas para datas

- A função **`strtotime()`** converte uma string para o tipo data
  - Exemplo: **`strtotime`**`('2019-04-01')`
- A função `date()` é utilizada para formatar a data e hora atual
  - **`date('d/m/Y')`** => 07/04/2019
- Existem vários parâmetros de formatação
  - d: dia do mês
  - m: representação numérica do mês
  - Y: ano com 4 dígitos

https://www.php.net/manual/pt_BR/function.date.php

Algumas funções prontas para datas

- Imprimir a data atual em português:

```
setlocale(LC_ALL, "pt_BR", "pt_BR.iso-8859-1", "pt_BR.utf-8", "portuguese");
date_default_timezone_set('America/Sao_Paulo');

echo strftime("%A, %d de %B de %Y", strtotime('today'));
```

Outras funções prontas

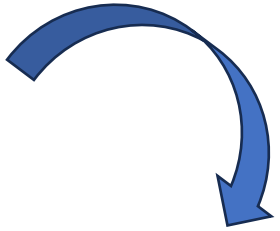
- `sqrt`: raiz quadrada
- `rand`: gera um inteiro aleatório
- `round`: arredonda um número
- `pow`: calcula o valor de uma potência
- `header('Location: https://www.google.com')`: Redireciona o usuário para google.com

https://www.php.net/manual/pt_BR/functions.user-defined.php

Classes e objetos

```php
<?php
class Fruta {
        // Propriedades
        public $nome;
        public $cor;

        // Métodos
        function set_nome($name) {
          $this->nome = $name;
        }
        function get_nome() {
          return $this->nome;
        }
}
?>
```



```php
<?php
    $melancia = new Fruta();
    $melancia->set_nome("Melancia");
?>
```

Importação de script dentro de outros

- É possível modularizar o código PHP para reutilização
- Dependendo da forma de inclusão, o script incluído pode ser obrigatório para continuar a execução de quem requisitou ou não
  - include : inclui um script PHP dentro de outro. Se o arquivo do script não for encontrado ou não funcionar, o servidor gera um *warning*, mas o código que estiver abaixo da chamada include continua sendo executado
  - require: inclui um script PHP dentro de outro. Se o arquivo do script não for encontrado ou não funcionar, o servidor gera um *erro fatal* e para de executar a aplicação
  - include_once e require_once : similares, porém incluem o script desejado somente uma vez

```php
<?php
    include "menu.php";
?>
```

Exercícios

1. Imprimir os números inteiros de 100 a 50 (ordem decrescente).
2. Altere o exercício 1 para colocar os números em uma lista não ordenada no html (ul).
3. Crie um array com 10 números e imprima a soma dos positivos e a quantidade de negativos.

Exercícios

4. Crie um array para armazenar o nome e a idade de 5 pessoas. As chaves devem ser os nomes das pessoas e os valores devem ser as respectivas idades. Imprima os nomes e as idades das pessoas.
5. Com o mesmo exercício anterior:
   - ordene o array pela idade (ordem crescente) antes de imprimir.
   - ordene o array pelo nome (ordem crescente) antes de imprimir.

Referências

- https://www.php.net/docs.php
- LOCKHART, J. e STURGEON, P. PHP: The Right Way. Lean Pub, 2016. Tradução disponível em http://br.phptherightway.com
- https://www.w3schools.com/php/